most-exchange FIX

Rules of Engagement

Connecting to most-exchange over FIX 4.4: the order-entry and market-data sessions, the messages and tags they accept and send, the order types the exchange supports, and how to diagnose a session that misbehaves. For a participant's FIX developers and support staff.

Revision 2026-11-01

FIX 4.4, with three custom tags (2362, 2964, 5001) and one tag from FIX 5.0 (969).

Examples are real messages from the development stack, the password masked as the adapter logs it.

© 2026 P.M.Vrolijk. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

Contents

1 **Introduction**

1.1 Scope

1.2 The venue and its sessions

1.3 Conventions

1.4 Identifiers

1.5 What is not supported

1.6 How to read this document

2 **The FIX session**

2.1 Connecting

2.2 Logon (35=A)

2.3 Sequence numbers

2.4 Heartbeats and test requests

2.5 Resend requests and gap fills

2.6 Messages sent while you were logged out

2.7 Resetting (141=Y)

2.8 After the venue restarts

2.9 Logout (35=5)

2.10 Session-level reject (35=3)

3 **Order entry messages**

3.1 Overview

3.2 Standard header and trailer

3.3 NewOrderSingle (35=D)

3.4 OrderCancelRequest (35=F)

3.5 OrderCancelReplaceRequest (35=G)

3.6 OrderStatusRequest (35=H)

3.7 OrderMassStatusRequest (35=AF)

3.8 ExecutionReport (35=8)

3.9 OrderCancelReject (35=9)

3.10 BusinessMessageReject (35=j)

3.11 Custom and non-standard tags

4 **Order types and lifecycle**

4.1 Limit orders only

4.2 Time in force

4.3 Self-match prevention

4.4 Order states

4.5 Cancelling

4.6 Asking for status

4.7 An order whose fate is unknown
4.8 Duplicates, resends and ClOrdID

5 **Market data**
5.1 The market-data session
5.2 The security list (35=x, 35=y)
5.3 MarketDataRequest (35=V)
5.4 The snapshot (35=W)
5.5 Updates (35=X)
5.6 MarketDataRequestReject (35=Y)
5.7 Resynchronisation and slow consumers

6 **Reject and reason codes**
6.1 Orders refused at the venue’s interface
6.2 Orders rejected by the exchange
6.3 Cancels refused (35=9)
6.4 Session-level rejects (35=3)
6.5 Business-level rejects (35=j)
6.6 Logouts
6.7 Unknown order on a status request
6.8 Market data

7 **Troubleshooting**
7.1 Before you call the venue
7.2 Cannot log on
7.3 Logged out during the session
7.4 Missing or duplicate reports
7.5 Orders and cancels refused
7.6 Market data
7.7 Testing your connection

8 **Index**
8.1 Tags
8.2 Messages
8.3 Terms

1. Introduction

1.1 Scope

This document is for a participant connecting to most-exchange over FIX: the developers who build the connection and the support staff who run it. It states what the venue's FIX interface accepts, what it sends, and what it guarantees. It does not repeat the FIX 4.4 specification; where this document is silent, FIX 4.4 applies.

The FIX interface is an adapter in front of most-exchange's matching engine. It adds no order types and no risk checks of its own: **an order is accepted only if the exchange can carry it**, and everything else is refused at the interface, with a reject that says why, before it reaches the market.

1.2 The venue and its sessions

Protocol	FIX 4.4 (`8=FIX.4.4`), with the custom tags of 3.11
Transport	TCP. TLS, where offered, is the venue operator's and is terminated before the FIX engine
Your role	Initiator. The venue is the acceptor
Sessions	Two per participant, each on its own port: **order entry** and **market data**

An order-entry session acts for one participant, the trading identity the venue configured for it. Every order and cancel on it is that participant's; nothing on a message can name another.

A market-data session asks for the list of securities and subscribes to their books. It carries no orders and acts for no participant.

The venue gives you, per session: the host and port, your SenderCompID, the venue's CompID, a username, and a password. The password is a generated secret: the venue keeps only a digest of it and can reissue it, never recover it.

1.3 Conventions

- **Prices and quantities are decimals, exact to eight places.** The exchange holds them as fixed-point integers with eight implied decimals; a value with more than eight decimals is refused, never rounded. `100.25`, `100.250`, `10` and `0.00000001` are all valid.
- **Timestamps** are UTC, `YYYYMMDD-HH:MM:SS.sss`, as FIX 4.4 defines UTCTimestamp.
- **Dates** (ExpireDate, 432) are LocalMktDate, `YYYYMMDD`.
- **A security** is named by SecurityID (48) with SecurityIDSource (22) `8`, the exchange's own numeric id, or `4`, its ISIN; or by Symbol (55) alone. The security list (section 5) gives all three.
- In tables, *Req* is **Y** for required, **N** for optional, **C** for conditional.
- Examples are real messages from the venue's development environment, with `|` for the SOH delimiter. Passwords appear as `***`, as the venue logs them.

1.4 Identifiers

Identifier	Tag	Whose	Rules
ClOrdID	11	Yours	1 to 20 characters of `A-Z a-z 0-9 - _ + ; : . /` (4.8). Unique per session for the trading day, and for as long as the order is open. Used on every order and cancel, and carried to the exchange as sent
OrigClOrdID	41	Yours	The ClOrdID of the order a cancel refers to
OrderID	37	The exchange's	Assigned when the order is acknowledged; `NONE` before that, and on a reject
ExecID	17	The exchange's	Unique per session. A report from the exchange carries the exchange's report sequence number; a message the interface answers itself carries `A<index>.<MsgSeqNum>` (3.8)

The exchange's ExecID never changes for a report: **a report delivered twice carries the same ExecID**, and you can drop the second (2.8).

1.5 What is not supported

- **Order types:** market, stop and stop-limit orders; IOC and FOK; anything but a limit order with time in force Day, GTC or GTD (section 4).
- **Replace:** OrderCancelReplaceRequest (35=G) is answered with a cancel reject (3.5).
- **Client fields the exchange does not carry**, Account (1) among them: accepted, ignored, not echoed.
- **Market by order:** market data is by price level (L1, L2) only.
- **Drop copy** and **trade capture:** not offered.

1.6 How to read this document

Section 2 is the FIX session: connecting, logging on, sequence numbers, and how nothing is lost across a disconnection. Section 3 is every order-entry message and tag, in and out. Section 4 is what an order can be and the states it goes through. Section 5 is market data. Section 6 collects every reject code and its text. Section 7 is troubleshooting. Section 8 indexes tags, messages and terms.

2. The FIX session

2.1 Connecting

Each session is one TCP connection to its own port: order entry at one, market data at another. **A session logs on only at its own port**; at the other it is refused like an unknown session. Connect, send a Logon, and wait for the venue's Logon before sending anything else.

If the connection drops, reconnect and log on again, with your sequence numbers as they were (2.3). Nothing sent to you while you were away is lost (2.6).

2.2 Logon (35=A)

Tag	Name	Req	Value
49	SenderCompID	Y	Your CompID, as the venue configured it
56	TargetCompID	Y	The venue's CompID
34	MsgSeqNum	Y	Your next outgoing number (2.3)
98	EncryptMethod	Y	`0`
108	HeartBtInt	Y	Seconds. The venue uses the same interval
141	ResetSeqNumFlag	N	`Y` to reset both sides to 1 (2.7). Order entry: only when you mean to. Market data: always, or log on at 1
553	Username	Y	As issued
554	Password	Y	As issued

A Logon is accepted only if SenderCompID, TargetCompID, Username and Password all match one session the venue configured. The venue answers with its own Logon:

```
8=FIX.4.4|9=093|35=A|49=CLIENT7|56=MOST|34=53|52=20261031-
18:55:03.218|98=0|108=10|141=N|553=client7|554=***|10=145|
8=FIX.4.4|9=76|35=A|49=MOST|56=CLIENT7|34=20052|52=20261031-18:55:03.232|98=0|108=10|141=N|10=032|
```

A refused Logon gets a Logout with Text `Logon refused`, at MsgSeqNum 1, and the connection is closed. It does not say what was wrong: a CompID, the username, the password, or the port are all the same refusal. Check each.

```
8=FIX.4.4|9=093|35=A|49=CLIENT7|56=MOST|34=1|52=20261031-
18:55:20.729|98=0|108=10|141=N|553=client7|554=***|10=252|
```

answered by `35=5 … 58=Logon refused`. Your session's state is untouched by a refused Logon.

2.3 Sequence numbers

Order-entry sessions are persistent. MsgSeqNum runs on across Logouts, disconnections and the venue's restarts, both ways, and resets only when you log on with ResetSeqNumFlag (141=Y). Keep your sequence store;

your first Logon on a new session is at 1.

- **Your Logon numbered above what the venue expects:** accepted. The venue then asks you to resend the gap (2.5).
- **Your Logon numbered below:** refused, with a Logout. Your sequence store and the venue's disagree; do not guess. Contact the venue, or reset (2.7).
- **The venue's Logon numbered above what you expect:** messages were sent to you while you were away. Ask for them with a ResendRequest (2.6).

> **NO END-OF-DAY RESET**
> The venue does not reset sequence numbers at the end of the day or log sessions out at the close. Sequence numbers run on until you reset them.

Market-data sessions are transient: both sides start at 1 at every Logon, and nothing is ever resent. Log on with 141=Y (or at 1) every time, and subscribe again (section 5).

2.4 Heartbeats and test requests

- Heartbeat (35=0) at the HeartBtInt you chose, both ways, when nothing else is sent.
- A TestRequest (35=1) is answered with a Heartbeat carrying its TestReqID (112).
- **Keep your clock synchronised.** SendingTime (52) more than two minutes from the venue's clock is a session reject, SessionRejectReason 10 (SendingTime accuracy problem).

2.5 Resend requests and gap fills

Both sides follow FIX 4.4's recovery:

- **When the venue asks you** (35=2), resend from BeginSeqNo to your last message sent; an EndSeqNo of 0 means that. Application messages go again with PossDupFlag (43) = Y and OrigSendingTime (122); session messages are gap-filled (35=4, GapFillFlag 123 = Y).
- **While your own ResendRequest is outstanding,** queue any message from the venue numbered above the range you asked for, and process it after the gap is filled. FIX requires it; an engine that instead counts it as received stalls its own recovery until it reconnects.
- **The venue asks for what you owe.** If messages from you are missing and its own recovery has not closed the gap within a second, the venue sends its own ResendRequest, and again each further second without progress. **After three without progress it logs you out**, with Text `messages <from> to <to> not received; resend them`, and asks again straight after your next Logon.

Resent orders are safe: a PossDup 35=D or 35=F the venue already has is dropped, never executed twice (4.8).

2.6 Messages sent while you were logged out

Reports do not wait for you. An execution report for an order of yours, a fill while you are logged out say, is sent to your session at once, numbered, and recorded by the venue. At your next Logon your engine sees the venue's MsgSeqNum above what it expects and asks for the gap; the reports come back with PossDupFlag = Y. As the venue's test client prints one:

```
ExecutionReport ClOrdID=S1793473103 OrderID=20030 ExecID=30 ExecType=TRADE
OrdStatus=PARTIALLY_FILLED Side=SELL Symbol=AAPL OrderQty=10 Price=100 LastQty=4 LastPx=100
CumQty=4 LeavesQty=6 AvgPx=100 PossDup
```

Process a PossDup report by its ExecID: if you have it, drop it; if not, it is new to you. Nothing in this path depends on how long you were away.

2.7 Resetting (141=Y)

A Logon with ResetSeqNumFlag = Y resets both sides' numbers to 1. **It also gives up every message you had not yet received**, reports included: the venue cannot resend across a reset.

After a reset on an order-entry session, **send an OrderMassStatusRequest (35=AF)** (3.7) and rebuild your orders from its answers. Each open order is stated with its current OrdStatus, CumQty and LeavesQty; fills you missed are reflected in CumQty, not reported one by one. An order that finished while you were not receiving (filled, cancelled, expired) is not open, and is not in the answer; ask for it by ClOrdID with an OrderStatusRequest (35=H) if you need its final state.

Reset only when you must: a lost sequence store, or the venue's own session state lost (2.8).

2.8 After the venue restarts

- **A planned restart** is invisible except as a disconnection: log on again with your numbers.
- **After a crash** the venue's outgoing MsgSeqNum jumps ahead by 10,000. You see a gap at the Logon, ask for it, and receive **one gap fill** covering the jump; carry on. The jump exists so that a message you received just before the crash is never numbered twice.
- **A report you had already received before a crash may come again**, under a new MsgSeqNum and without PossDupFlag, because the venue cannot tell which ones reached you. **Its ExecID is the same:** deduplicate execution reports by ExecID, always.
- **If the venue loses your session's state** (an operator's failure, not a restart), its sequence numbers are no longer the ones you hold. Log on with 141=Y and recover with a 35=AF (2.7). Your orders, their state and the ClOrdIDs you used that day are kept; reports the venue had not yet delivered to you are not resent.
- **No order is lost.** Every order that reached the venue before the crash is answered after the restart: it reached the exchange and is reported, or it did not and is rejected, or its fate is unknown and it is said so (4.7).

2.9 Logout (35=5)

Either side may log out; the other answers with a Logout. Text (58), when the venue sends it, says why: see section 6.3. Your orders stay in the market after a Logout or a disconnection: there is no cancel on disconnect. Reports for them are sent to your session while you are away (2.6).

2.10 Session-level reject (35=3)

A message that is malformed for FIX is answered with a Reject and **not processed**: an order rejected this way is not an order, and its ClOrdID is not used.

Tag	Name	Value
45	RefSeqNum	The rejected message's MsgSeqNum
372	RefMsgType	Its MsgType
371	RefTagID	The offending tag, when there is one
373	SessionRejectReason	Why: section 6.4

What draws one: a required tag missing, a value of the wrong type, a value outside the tag's allowed values (2964=3, say), a tag not defined for the message type, and a tag the venue's dictionary does not know at all. The venue's dictionary is FIX 4.4's, for the messages it handles, plus the tags of 3.11: **send only tags FIX 4.4 defines for the message, or those**.

3. Order entry messages

3.1 Overview

You send		The venue answers
NewOrderSingle	35=D	ExecutionReport (35=8): acknowledged, or rejected
OrderCancelRequest	35=F	35=8 ExecType 4, cancelled; or OrderCancelReject (35=9)
OrderCancelReplaceRequest	35=G	35=9: not supported
OrderStatusRequest	35=H	35=8 ExecType I
OrderMassStatusRequest	35=AF	One 35=8 ExecType I per open order, or BusinessMessageReject (35=j)
Any other application message		35=j, BusinessRejectReason 3, unsupported message type

Unsolicited, the venue sends 35=8 for every fill, cancellation, expiry and change of an order's status, and 35=9 when a cancel it accepted is refused by the exchange. The session messages, 35=0, 1, 2, 3, 4, 5 and A, are FIX 4.4's (section 2).

3.2 Standard header and trailer

Tag	Name	Req	Notes
8	BeginString	Y	`FIX.4.4`
9	BodyLength	Y	
35	MsgType	Y	
49	SenderCompID	Y	Yours
56	TargetCompID	Y	The venue's
34	MsgSeqNum	Y	Section 2.3
52	SendingTime	Y	UTC, within two minutes of the venue's clock
43	PossDupFlag	C	`Y` on a resent message
97	PossResend	N	
122	OrigSendingTime	C	Required with 43=Y
10	CheckSum	Y	

Other FIX 4.4 header fields are accepted and ignored.

3.3 NewOrderSingle (35=D)

Tag	Name	Req	Values and rules
11	ClOrdID	Y	1 to 20 characters of `A-Z a-z 0-9 - _ + ; : . /`, unique per session for the trading day and while the order is open (4.8)

Tag	Name	Req	Values and rules
55	Symbol	C	The security, if 48 is absent
48	SecurityID	C	The security: the exchange's id with 22=8, or its ISIN with 22=4. Takes precedence over 55
22	SecurityIDSource	C	Required with 48: `8` (exchange id) or `4` (ISIN)
54	Side	Y	`1` buy, `2` sell
38	OrderQty	Y	Greater than 0, exact to eight decimals
40	OrdType	Y	`2`, limit, only
44	Price	Y	Exact to eight decimals, and on the security's tick (MinPriceIncrement, 969, in the security list). Price collars are the exchange's to check (6.2)
59	TimeInForce	N	`0` day (the default), `1` good till cancel, `6` good till date
432	ExpireDate	C	Required with 59=6. `YYYYMMDD`; a date already past is rejected by the exchange
60	TransactTime	Y	Required by FIX 4.4; not forwarded
2362	SelfMatchPreventionID	N	A positive integer, as text. Absent: your participant (4.3)
2964	SelfMatchPreventionInstruction	N	`1` cancel the aggressor (the default), `2` cancel the resting order

Other FIX 4.4 fields of 35=D, Account (1) and HandlInst (21) among them, are accepted and ignored.

Answered by a 35=8: ExecType 0, *new*, once the exchange has the order; or ExecType 8, *rejected*, by the venue's interface (section 6.1) or by the exchange (6.2). An order refused for what it says (6.1, the first table) is answered at once, never reaches the market, and its ClOrdID is not used up; one refused because the venue cannot trade (`exchange unavailable`, 6.1) has used its ClOrdID: send it again under a new one.

```
8=FIX.4.4|9=128|35=D|49=CLIENT7|56=MOST|34=54|52=20261031-
18:55:03.240|11=RB1793472897|55=AAPL|54=1|60=20261031-18:55:02.865|38=25|40=2|44=99.5|10=171|
8=FIX.4.4|9=183|35=8|49=MOST|56=CLIENT7|34=20053|52=20261031-
18:55:03.265|37=20029|11=RB1793472897|17=27|150=0|39=0|55=AAPL|48=1|22=8|54=1|38=25|44=99.5|59=0|15
1=25|14=0|6=0|60=20261031-18:55:03.266|10=054|
```

Refused at the interface, an IOC and an unknown security:

```
8=FIX.4.4|9=133|35=D|49=CLIENT7|56=MOST|34=57|52=20261031-
18:55:05.746|11=RI1793472897|55=AAPL|54=1|60=20261031-18:55:05.376|38=25|40=2|44=99.5|59=3|10=157|
8=FIX.4.4|9=216|35=8|49=MOST|56=CLIENT7|34=20056|52=20261031-
18:55:05.750|37=NONE|11=RI1793472897|17=A0.20056|150=8|39=8|103=99|55=AAPL|54=1|38=25|44=99.5|59=3|
151=0|14=0|6=0|60=20261031-18:55:05.751|58=time in force 0, 1 or 6 only|10=150|
8=FIX.4.4|9=194|35=8|49=MOST|56=CLIENT7|34=20059|52=20261031-
18:55:08.262|37=NONE|11=RZ1793472897|17=A0.20059|150=8|39=8|103=1|55=ZZZZ|54=1|38=1|44=1|151=0|14=0
|6=0|60=20261031-18:55:08.263|58=unknown security|10=209|
```

3.4 OrderCancelRequest (35=F)

Tag	Name	Req	Values and rules
11	ClOrdID	Y	The cancel's own id, of the same characters and unique as an order's is (4.8)
41	OrigClOrdID	Y	The order's ClOrdID

Tag	Name	Req	Values and rules
37	OrderID	N	If sent, must be the order's
54	Side	Y	Must be the order's
55, 48, 22	The security	N	If sent, must be the order's
60	TransactTime	Y	

The venue finds the order by OrigClOrdID; anything you send besides must match it, or the cancel is refused (35=9, CxlRejReason 1). Other FIX 4.4 fields of 35=F, OrderQty (38) among them, are accepted and not used.

Answered by a 35=8 for the order, ExecType 4, *cancelled*, with 11 the cancel's ClOrdID and 41 the order's; or by a 35=9 (3.9).

```
8=FIX.4.4|9=131|35=F|49=CLIENT7|56=MOST|34=69|52=20261031-
18:55:15.763|41=RB1793472897|11=RC1793472897|55=AAPL|54=1|60=20261031-18:55:15.388|38=25|10=192|
8=FIX.4.4|9=198|35=8|49=MOST|56=CLIENT7|34=20068|52=20261031-
18:55:15.778|37=20029|11=RC1793472897|41=RB1793472897|17=28|150=4|39=4|55=AAPL|48=1|22=8|54=1|38=25
|44=99.5|59=0|151=0|14=0|6=0|60=20261031-18:55:15.779|10=125|
```

- **A cancel sent before the order is acknowledged** is held and sent as soon as it is. If the order's first report is final (rejected, or filled at once), the cancel is refused *too late*.
- **One cancel at a time:** a cancel for an order with another cancel still pending is refused, CxlRejReason 3. Once the first is answered, a new one is accepted.

3.5 OrderCancelReplaceRequest (35=G)

Not supported: the exchange has no replace. Answered with a 35=9, CxlRejResponseTo (434) 2, CxlRejReason (102) 2 and Text `replace not supported`. The order is unchanged. To change an order, cancel it and enter a new one.

3.6 OrderStatusRequest (35=H)

Tag	Name	Req	Values and rules
11	ClOrdID	Y	The order's ClOrdID: the order is found by this alone
54	Side	Y	
55, 48, 22	The security	C	As FIX 4.4 requires; echoed on an unknown order
790	OrdStatusReqID	N	Echoed on the answer

Answered with a 35=8, ExecType I, stating the order as you were last told it: OrdStatus, CumQty, LeavesQty, AvgPx. A ClOrdID of a cancel gets its order's status, with 11 the cancel's and 41 the order's.

```
8=FIX.4.4|9=84|35=H|49=CLIENT7|56=MOST|34=63|52=20261031-
18:55:10.763|11=RB1793472897|55=AAPL|54=1|10=079|
8=FIX.4.4|9=189|35=8|49=MOST|56=CLIENT7|34=20062|52=20261031-
18:55:10.767|37=20029|11=RB1793472897|17=A0.20062|150=I|39=0|55=AAPL|48=1|22=8|54=1|38=25|44=99.5|5
9=0|151=25|14=0|6=0|60=20261031-18:55:10.768|10=143|
```

An unknown ClOrdID (never sent, or no longer held after its trading day) is answered as FIX 4.4 says: ExecType I, OrdStatus 8, OrdRejReason 5, Text `unknown order`, OrderID `NONE`, and ClOrdID, Side and the security echoed.

3.7 OrderMassStatusRequest (35=AF)

Tag	Name	Req	Values and rules
584	MassStatusReqID	Y	Echoed on every answer
585	MassStatusReqType	Y	7 all your orders, or 1 one security. Any other value is a session reject (6.4)
55, 48, 22	The security	C	Required with 585=1
54	Side	N	Only orders of this side

Answered with one 35=8, ExecType I, per open order, each with MassStatusReqID (584), TotNumReports (911) and LastRptRequested (912), `Y` on the last. *Open* is every order not in a final state: pending, new, partially filled, or of unknown status (4.7).

```
8=FIX.4.4|9=222|35=8|49=MOST|56=CLIENT7|34=20065|52=20261031-
18:55:13.273|37=20029|11=RB1793472897|584=MS-
1793472912902|911=1|912=Y|17=A0.20065|150=I|39=0|55=AAPL|48=1|22=8|54=1|38=25|44=99.5|59=0|151=25|1
4=0|6=0|60=20261031-18:55:13.274|10=016|
```

Answered instead with a 35=j:

Case	BusinessRejectReason (380)	Text
No open order matches	0	`no open orders`
585=1 with no security named	5	
585=1 naming a security the venue does not list	2	`unknown security`
The venue not ready to trade	4	`exchange unavailable`

3.8 ExecutionReport (35=8)

Tag	Name	Notes
37	OrderID	The exchange's order id; `NONE` before it is acknowledged
11	ClOrdID	The order's; on the report of a cancel, the cancel's
41	OrigClOrdID	On the report of a cancel: the order's ClOrdID
17	ExecID	See below
150	ExecType	`0` new, `F` trade, `4` cancelled, `C` expired, `8` rejected, `I` order status
39	OrdStatus	See the table below
55, 48, 22	The security	Symbol, the exchange's SecurityID, SecurityIDSource 8
54	Side	As sent
38	OrderQty	As sent
44	Price	As sent

Tag	Name	Notes
59, 432	TimeInForce, ExpireDate	As sent; 59 stated on every order
2362, 2964	Self-match prevention	Echoed when sent
32, 31	LastQty, LastPx	On a trade
151	LeavesQty	Stated by the exchange
14	CumQty	Stated by the exchange
6	AvgPx	Over the fills reported to you; 0 before any
60	TransactTime	The venue's time when it reported
103, 58	OrdRejReason, Text	On a reject (section 6)
5001	OrdStatusUnknown	`Y` when the order's status is unknown (4.7); absent otherwise
584, 911, 912	Mass status	On an answer to 35=AF
43	PossDupFlag	`Y` on a resend (2.6)

ExecType (150)	OrdStatus (39)
`0` new	`0` new
`F` trade	`1` partially filled, or `2` filled when LeavesQty is 0
`4` cancelled	`4` cancelled
`C` expired	`C` expired
`8` rejected	`8` rejected
`I` order status	The order's: `A` pending new, `0`, `1`, `2`, `4`, `C` or `8`

ExecID. A report from the exchange carries the exchange's report sequence number, in decimal; it never changes for that report, so a duplicate is recognised by it. A message the venue's interface answers itself (a reject at the interface, a status, an unknown status) carries `A<index>.<MsgSeqNum>`, the MsgSeqNum of the 35=8 itself; those are never sent twice.

CumQty and LeavesQty are the exchange's own figures on every report, never derived; trust them over a sum of fills. A status that finds CumQty above what you were told means fills were made that could not be reported one by one: Text says `fills not reported individually: CumQty <a> to <b>, prices unknown`, and AvgPx stays that of the fills you were told of.

3.9 OrderCancelReject (35=9)

Tag	Name	Notes
37	OrderID	The order's; `NONE` if unknown or not yet acknowledged
11	ClOrdID	The cancel's (or the replace's)
41	OrigClOrdID	As sent
39	OrdStatus	The order's status as you were last told it; `8` when the order is unknown (CxlRejReason 1)
434	CxlRejResponseTo	1 to a 35=F, 2 to a 35=G
102	CxlRejReason	Section 6.3

Tag	Name	Notes
58	Text	Why, in words
5001	OrdStatusUnknown	`Y` when the order's status is unknown

```
8=FIX.4.4|9=131|35=F|49=CLIENT7|56=MOST|34=72|52=20261031-
18:55:18.265|41=RB1793472897|11=RD1793472897|55=AAPL|54=1|60=20261031-18:55:17.892|38=25|10=189|
8=FIX.4.4|9=138|35=9|49=MOST|56=CLIENT7|34=20071|52=20261031-
18:55:18.269|37=20029|11=RD1793472897|41=RB1793472897|39=4|434=1|102=0|58=too late to
cancel|10=152|
```

(A second cancel of an order already cancelled: *too late*, OrdStatus 4.)

3.10 BusinessMessageReject (35=j)

Tag	Name	Notes
45	RefSeqNum	The refused message's MsgSeqNum
372	RefMsgType	Its MsgType
380	BusinessRejectReason	`3` unsupported message type; for 35=AF, section 3.7
58	Text	Why, in words

A 35=j you send is never answered.

3.11 Custom and non-standard tags

Tag	Name	Type	On	Source
2362	SelfMatchPreventionID	String: a positive integer	35=D, echoed on 35=8	FIX Latest (EP211)
2964	SelfMatchPreventionInstruction	int: 1 or 2	35=D, echoed on 35=8	FIX Latest (EP280); its value 3 is not accepted
5001	OrdStatusUnknown	Boolean	35=8, 35=9	The venue's own (user-defined range)
969	MinPriceIncrement	Float	35=y entries	FIX 5.0; the tick size

4. Order types and lifecycle

4.1 Limit orders only

The exchange matches **limit orders** on one book per security, by price and then time. An order is OrdType (40) `2` with a Price (44). The exchange runs trading phases (pre-open, an opening auction, continuous trading, closed); an order outside the phases that accept it is rejected `MARKET_CLOSED` (OrdRejReason 2).

Refused at the venue's interface, before the market, with OrdRejReason 99 and a fixed Text (section 6.1):

Not supported	Text
Market, stop and any OrdType but 2	`limit orders only`
IOC (59=3), FOK (59=4), at the open or close (59=2, 7), GTX	`time in force 0, 1 or 6 only`
Replace (35=G)	`replace not supported`, on a 35=9

4.2 Time in force

TimeInForce (59)	Lives until
`0` Day, or absent	The end of the trading day. The venue stamps it with the venue's trading date when it arrives; it is reported expired (ExecType C) when the exchange purges the day's orders
`1` Good till cancel	Cancelled or filled; kept across days
`6` Good till date	The end of ExpireDate (432), `YYYYMMDD`. A date already past is rejected `ORDER_EXPIRED`

4.3 Self-match prevention

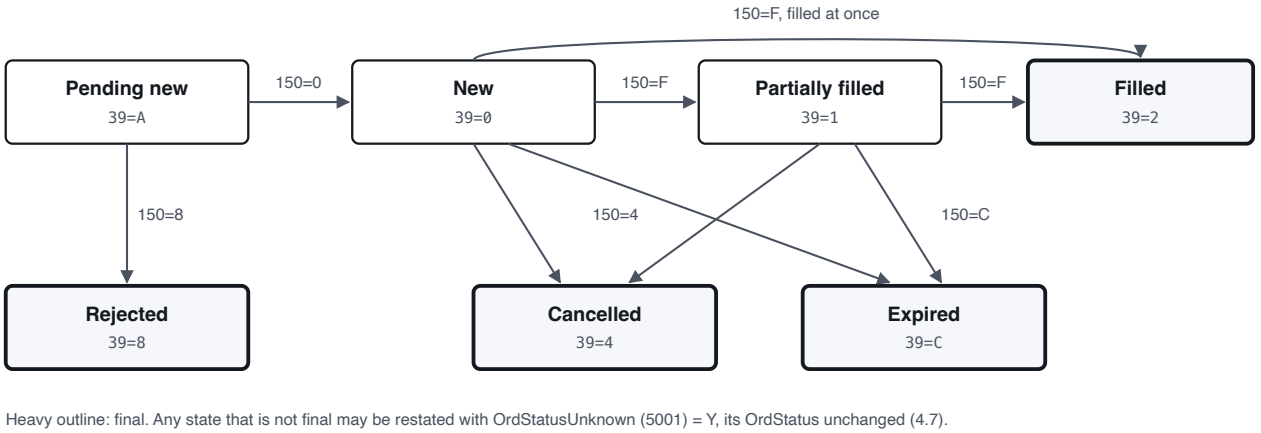
The exchange does not let an order trade with another order of the same **SMP id**. An order's SMP id is its SelfMatchPreventionID (2362) if sent, otherwise your participant: **by default your own orders never trade with each other**.

When an incoming order would match one with the same SMP id, the incoming order's SelfMatchPreventionInstruction (2964) decides:

2964	Effect
1, or absent	Cancel the incoming order's remainder; the resting order stays
2	Cancel the resting order and go on matching

The order cancelled gets a 35=8 ExecType 4 with Text `SELF_MATCH_PREVENTED`. A self-match prints no trade. Give different SMP ids to strategies that may trade with each other: two orders with different SelfMatchPreventionIDs match normally.

4.4 Order states



Heavy outline: final. Any state that is not final may be restated with OrdStatusUnknown (5001) = Y, its OrdStatus unchanged (4.7).

Figure 4.1 An order's states, as OrdStatus (39) reports them

OrdStatus (39)	Means	Final
`A` Pending new	Received by the venue, not yet acknowledged by the exchange. Seen on a status answer or a cancel reject	No
`0` New	Resting in the book	No
`1` Partially filled	Resting, partly filled	No
`2` Filled	Fully filled	Yes
`4` Cancelled	Cancelled by you, by self-match prevention, by a volatility halt or by the exchange's operator	Yes
`C` Expired	A day or GTD order past its date	Yes
`8` Rejected	Refused, by the venue's interface or by the exchange	Yes

Status unknown is not a state of its own: an order whose fate the venue cannot tell keeps the OrdStatus last reported, flagged with OrdStatusUnknown (5001) = Y (4.7).

4.5 Cancelling

- **By you:** an OrderCancelRequest (3.4). The answer is the order's 35=8, ExecType 4, or a 35=9 saying why not (6.3).
- **Unsolicited**, a 35=8 ExecType 4 for the order, Text naming the reason:

Text	Why
`SELF_MATCH_PREVENTED`	Self-match prevention (4.3)
`VOLATILITY_HALT`	The incoming order's remainder, at a dynamic collar breach
absent	Cancelled by the exchange's operator

- **No cancel on disconnect.** Your orders stay in the book when you log out or lose the connection.

4.6 Asking for status

- **One order:** OrderStatusRequest (35=H) by ClOrdID (3.6).
- **All open orders, or those of one security:** OrderMassStatusRequest (35=AF) (3.7). Use it after a reset (2.7) or whenever your view and the venue's may differ.

Both are answered from the venue's record of what it has told you: they never contradict the reports you received.

4.7 An order whose fate is unknown

When the exchange's gateway fails over, the venue settles every order in flight with the exchange's own resend:

- **An order that never reached the exchange** is rejected, OrdRejReason 99, Text `not sequenced (gateway failover)`. It did not trade. You may send it again, under a new ClOrdID.
- **An order whose fate the resend cannot tell** is not open at the exchange, but may have traded before it stopped being open. The venue **never reports it as rejected, cancelled, expired or done**, since you might then enter its quantity again. It restates it, ExecType I, with the OrdStatus, CumQty and LeavesQty you were last told, and:
  - OrdStatusUnknown (5001) = `Y`;
  - Text `STATUS UNKNOWN: not open at the exchange; fills since CumQty=<n> unrecoverable until reconciled`.

What to do with an unknown order: treat it as not working, and its fills since the CumQty stated as unknown. Do not resend it as if it had not traded. Ask the venue's operations to reconcile it. A later report for it, if one arrives, clears the flag. A cancel for it is answered at once with a 35=9, CxlRejReason 99 and the same flag.

A cancel in flight at a failover is settled the same way: never sequenced, a 35=9 CxlRejReason 99 `not sequenced` (the order is unchanged); unknown, a 35=9 with OrdStatusUnknown = Y and Text `cancel status unknown`.

4.8 Duplicates, resends and ClOrdID

- **A ClOrdID is 1 to 20 characters**, each one of `A-Z a-z 0-9 - _ + ; : . /`, on an order and on a cancel. It is carried to the exchange as you sent it, and comes back as you sent it on every report. One that does not fit, a 36-character UUID among them, is rejected at once and never reaches the market: a 35=D with OrdRejReason 99, a 35=F with a 35=9, CxlRejReason 99, both with Text `ClOrdID must be 1-20 of A-Z a-z 0-9 - _ + ; : . /`. A cancel whose OrigClOrdID does not fit names no order: 35=9, CxlRejReason 1, `unknown order`.
- **A new 35=D with a ClOrdID the session already used** for an order still open, or that day, is rejected, OrdRejReason 6, Text `duplicate ClOrdID`. A cancel's ClOrdID counts as used too.
- **A resent 35=D or 35=F (PossDupFlag = Y) the venue already has** is not executed again. If it was answered, the resend is dropped and the answer is not repeated; if not, it is answered once.
- **The same message delivered twice**, by your engine's resend or the venue's own recovery, is recognised by its MsgSeqNum and processed once.

5. Market data

5.1 The market-data session

A separate session, at its own port, with its own CompIDs and credentials. It is **transient**: both sides start at MsgSeqNum 1 at every Logon, nothing is resent, and subscriptions end with the session. After a reconnect, log on with ResetSeqNumFlag = Y and subscribe again: you get a fresh snapshot, never a stale one.

Market data is **by price level**: the best bid and offer (MarketDepth 1), or every level (0), plus last trades. There is no market by order.

You send		The venue answers
SecurityListRequest	35=x	SecurityList (35=y), one or more
MarketDataRequest	35=V	MarketDataSnapshotFullRefresh (35=W), then MarketDataIncrementalRefresh (35=X); or MarketDataRequestReject (35=Y)

5.2 The security list (35=x, 35=y)

Tag	Name	Req	Values
320	SecurityReqID	Y	Yours, echoed
559	SecurityListRequestType	Y	`4`, all securities. Any other is answered with SecurityRequestResult 1

The answer lists every security, in SecurityID order, **at most 100 per 35=y**:

Tag	Name	Notes
320	SecurityReqID	Yours
322	SecurityResponseID	Your SecurityReqID
560	SecurityRequestResult	`0` the list; `1` invalid or unsupported request; `4` temporarily unavailable, before the venue has its directory
393	TotNoRelatedSym	The number of securities in all the messages
893	LastFragment	`Y` on the last message
146	NoRelatedSym	Then, per security:
55	Symbol	
48, 22	SecurityID, SecurityIDSource	The exchange's id, `8`
454	NoSecurityAltID	`1`, then:
455, 456	SecurityAltID, SecurityAltIDSource	The ISIN, `4`
107	SecurityDesc	The security's name
15	Currency	
969	MinPriceIncrement	The tick size (FIX 5.0's tag)

```
8=FIX.4.4|9=79|35=x|49=FEED7|56=MOST|34=2|52=20261031-18:55:21.282|320=SL-
1793472920856|559=4|10=023|
8=FIX.4.4|9=285|35=y|49=MOST|56=FEED7|34=2|52=20261031-18:55:21.288|320=SL-1793472920856|322=SL-
1793472920856|560=0|393=2|893=Y|146=2|55=AAPL|48=1|22=8|454=1|455=US0378331005|456=4|107=Apple
Inc.|15=USD|969=0.01|55=MSFT|48=2|22=8|454=1|455=US5949181045|456=4|107=Microsoft
Corporation|15=USD|969=0.01|10=029|
```

5.3 MarketDataRequest (35=V)

Tag	Name	Req	Values
262	MDReqID	Y	Your subscription's name; unique among your session's subscriptions
263	SubscriptionRequestType	Y	`0` snapshot only; `1` snapshot plus updates; `2` unsubscribe
264	MarketDepth	Y	`1` top of book; `0` every level
265	MDUpdateType	C	Required with 263=1: `1`, incremental
266	AggregatedBook	N	`Y` or absent. `N` is refused
267	NoMDEntryTypes	Y	Then 269 MDEntryType, once per type: `0` bid, `1` offer, `2` trade
146	NoRelatedSym	Y	Then, per security: 48 with 22 (`8` or `4`), or 55

A subscription may name several securities; each gets its own 35=W and its own 35=X. **263=2** removes the subscription named by MDReqID, unanswered; an MDReqID not subscribed is ignored. **263=0** sends each security's 35=W and keeps nothing.

Refused, with a 35=Y (5.6): TradingSessionID (336), Scope (546), OpenCloseSettlFlag (286), and MDImplicitDelete (547) = Y.

```
8=FIX.4.4|9=129|35=V|49=FEED7|56=MOST|34=2|52=20261031-18:55:23.776|262=MD-
1793472923401|263=1|264=1|265=1|267=3|269=0|269=1|269=2|146=1|55=AAPL|10=051|
```

5.4 The snapshot (35=W)

One per security, first in every subscription:

Tag	Name	Notes
262	MDReqID	Yours
55, 48, 22	The security	Symbol, SecurityID, `8`
268	NoMDEntries	Then the bids, best first; the offers, best first; then the last trade, if you asked for trades and there is one
269	MDEntryType	`0` bid, `1` offer, `2` trade
270	MDEntryPx	The level's price
271	MDEntrySize	The level's total quantity
346	NumberOfOrders	The orders at the level (not on a trade)

One level per side at MarketDepth 1, every level at 0. An empty book is a 35=W with no entries; only the entry types you asked for are sent.

```
8=FIX.4.4|9=134|35=W|49=MOST|56=FEED7|34=2|52=20261031-18:55:23.781|262=MD-
1793472923401|55=AAPL|48=1|22=8|268=1|269=0|270=99.98|271=0.00000089|346=1|10=079|
```

(The book had one bid, no offer and no trade yet.)

5.5 Updates (35=X)

One 35=X per change, with the security on each entry:

Tag	Name	Notes
262	MDReqID	Yours
268	NoMDEntries	Then, per entry:
279	MDUpdateAction	`0` new, `1` change, `2` delete
269	MDEntryType	`0` bid, `1` offer, `2` trade
55, 48, 22	The security	
270	MDEntryPx	The level's price
271	MDEntrySize	Its new total quantity; absent on a delete
346	NumberOfOrders	Its orders; absent on a delete

At MarketDepth 0 (every level): `0` a price the book did not have; `1` a new total at a price it had; `2` the level is gone.

At MarketDepth 1 (top of book), you hold one entry per side: a new best price is a `2` for the old one and a `0` for the new; a new total at the same best price is a `1`; an empty side is a `2`. A change below the best sends nothing.

A trade is 269=2, 279=0, with 270 and 271.

5.6 MarketDataRequestReject (35=Y)

Tag	Name	Notes
262	MDReqID	Yours
281	MDReqRejReason	Below
58	Text	What was refused

281	Why
`0`	A security the venue does not list. The whole request is refused
`1`	The MDReqID is already subscribed on this session
`5`	MarketDepth other than 0 or 1
`6`	MDUpdateType other than 1, or none, with 263=1
`7`	AggregatedBook = N

281	Why
8	An MDEntryType other than 0, 1 or 2
9	A TradingSessionID
A	A Scope
B	An OpenCloseSettlFlag
C	MDImplicitDelete = Y

5.7 Resynchronisation and slow consumers

- **A book the venue is not sure of**, when it has lost sequence with the exchange's feed, sends you nothing: your snapshot waits, updates stop. Once the book is trusted again, you get a **fresh 35=W** that replaces what you hold, and updates resume. If the book is exactly as you hold it, updates simply resume, with no 35=W.
- **If you cannot keep up**, the venue does not queue updates without limit: it stops sending you 35=X for that subscription and sends a **fresh 35=W** as soon as it can. Your book is replaced, never left wrong; you are not logged out.
- **Apply a 35=W as a replacement**, not as changes, every time one arrives, not only the first.

6. Reject and reason codes

Every refusal the venue sends, with its code and its Text. **The Texts are fixed**: you may match on them.

6.1 Orders refused at the venue's interface

A 35=8, ExecType 8, OrdStatus 8, OrderID `NONE`, answered at once; the order never reaches the market. For those in the first table nothing is recorded, and the ClOrdID may be used again.

Refused	OrdRejReason (103)	Text (58)
11 not 1 to 20 of `A-Z a-z 0-9 - _ + ; : . /` (4.8)	99	`ClOrdID must be 1-20 of A-Z a-z 0-9 - _ + ; : . /`
Neither 48 nor 55; 48 without 22, or 22 other than 4 or 8; a security the venue does not list	1	`unknown security`
38 absent, zero or negative, or not exact at 8 decimals	13	`incorrect quantity`
40 other than 2	99	`limit orders only`
44 absent	99	`price required`
44 not exact at 8 decimals	99	`price not exact at 8 decimals`
54 other than 1 or 2	99	`side 1 or 2 only`
59 other than 0, 1 or 6	99	`time in force 0, 1 or 6 only`
59=6 without 432, or 432 not a `YYYYMMDD` date	99	`ExpireDate required for GTD`
2362 not a positive integer	99	`SelfMatchPreventionID must be a positive integer`

Refused after the order was recorded (its ClOrdID is used):

Refused	OrdRejReason	Text
ClOrdID already used on the session (4.8)	6	`duplicate ClOrdID`
The venue cannot reach the exchange, or is not yet ready to trade	99	`exchange unavailable`
The exchange's gateway does not serve your participant	99	`participant not served`
Never reached the exchange, at a gateway failover (4.7)	99	`not sequenced (gateway failover)`

6.2 Orders rejected by the exchange

A 35=8, ExecType 8, with **the exchange's reason as Text**:

Text	OrdRejReason	Why
`UNKNOWN_SECURITY`	1	The security is not traded on the exchange's shard

Text	OrdRejReason	Why
`MARKET_CLOSED`	2	The security is not in a phase that accepts orders; a volatility halt looks like this
`PRICE_OUT_OF_BOUNDS`	99	Outside the static price collar
`PRICE_OUT_OF_LADDER`	99	Outside the book's price range
`BOOK_CAPACITY`	99	The book cannot take more orders
`ORDER_EXPIRED`	99	ExpireDate before the current trading date
`UNAUTHORIZED_PARTICIPANT`	99	Your participant may not trade through the venue's gateway; contact the venue

6.3 Cancels refused (35=9)

Refused	CxlRejReason (102)	Text (58)
The order is already filled, cancelled, expired or rejected	0	`too late to cancel`
The cancel's own 11 not 1 to 20 of `A-Z a-z 0-9 - _ + ; : . /` (4.8)	99	`ClOrdID must be 1-20 of A-Z a-z 0-9 - _ + ; : . /`
No order with that OrigClOrdID, 41 names a cancel, or 41 does not fit (4.8)	1	`unknown order`
OrderID, Side or the security sent and not the order's	1	`does not match the order`
A replace (35=G), CxlRejResponseTo 2	2	`replace not supported`
Another cancel of the order is pending	3	`already pending cancel`
The exchange refused it: the order is not resting	1	`UNKNOWN_ORDER` (or 0, `too late to cancel`, when the venue already knows the order is finished)
The exchange refused it: the market is closed	2	`MARKET_CLOSED`
The venue cannot reach the exchange	99	`exchange unavailable`
Never reached the exchange, at a failover	99	`not sequenced`
Its fate unknown, at a failover; OrdStatusUnknown = Y	99	`cancel status unknown`
The order's status is unknown (4.7); OrdStatusUnknown = Y	99	`STATUS UNKNOWN: …`

6.4 Session-level rejects (35=3)

The message is not processed (2.10). SessionRejectReason (373), FIX 4.4's codes; those you will meet:

373	Means
`0`	Invalid tag number: a tag the venue's dictionary does not define
`1`	Required tag missing

373	Means
2	Tag not defined for this message type
5	Value incorrect (out of range) for this tag: 2964=3, 585 other than 1 or 7
6	Incorrect data format for value
10	SendingTime accuracy problem: your clock (2.4)

RefTagID (371) names the tag when there is one.

6.5 Business-level rejects (35=j)

380	Text (58)	When
0	`no open orders`	A 35=AF that matches no open order
2	`unknown security`	A 35=AF naming a security the venue does not list
3	`unsupported message type`	A message type the venue does not handle
4	`exchange unavailable`	A 35=AF before the venue is ready to trade
5	`security required for MassStatusReqType 1`	A 35=AF of type 1 with no security

6.6 Logouts

Text (58)	Why	What to do
`Logon refused`	Your Logon matched no session: CompIDs, username, password, or the port (2.2)	Check each; the venue does not say which
`messages <from> to <to> not received; resend them`	Three ResendRequests went unanswered (2.5)	Resend that range after your next Logon

6.7 Unknown order on a status request

A 35=H for a ClOrdID the venue does not hold: 35=8, ExecType I, OrdStatus 8, OrdRejReason 5, Text `unknown order`, OrderID `NONE` (3.6).

6.8 Market data

MDReqRejReason (281) is in 5.6; SecurityRequestResult (560) in 5.2.

7. Troubleshooting

7.1 Before you call the venue

The venue records every FIX message on your session, both ways. To find yours quickly, give:

- the session: your SenderCompID, and whether order entry or market data;
- the **MsgSeqNums** involved, yours and the venue's, and the times in UTC;
- for an order: its **ClOrdID**, and the OrderID and ExecIDs you received;
- the raw messages from your engine's log, if you have them.

7.2 Cannot log on

What you see	Likely cause	What to do
Connection refused	Wrong host or port; the venue is not accepting connections (a restart takes some seconds)	Check the address; retry after a few seconds
Logout `Logon refused`, at MsgSeqNum 1	One of SenderCompID, TargetCompID, Username, Password does not match, **or you connected to the other session's port**	Check all four and the port. The venue does not say which
Logout naming a MsgSeqNum too low	Your Logon is numbered below what the venue expects: your sequence store was reset or restored	Do not guess a number. Contact the venue, or reset with 141=Y and recover with 35=AF (2.7)
Market data: Logon refused or out of sequence after a reconnect	Market-data sessions start at 1 every time	Log on with 141=Y, or at MsgSeqNum 1

7.3 Logged out during the session

Text	What to do
`messages <from> to <to> not received; resend them`	Your engine did not answer the venue's ResendRequests. After your next Logon, resend that range, PossDup = Y. Check that your engine answers a ResendRequest with EndSeqNo 0 up to its last message sent (2.5)
No Text, after a period of silence	Heartbeats missed: your engine stopped sending, or the network did. Check HeartBtInt and that your engine answers TestRequests
A session reject before it, SessionRejectReason 10	Your clock is more than two minutes off

7.4 Missing or duplicate reports

- **A fill you did not receive.** If it happened while you were logged out, your engine should have seen the venue's MsgSeqNum jump at the next Logon and asked for the gap (2.6). If it did not ask, or skipped the gap, the report is still in the venue's record: ask for the order's status (35=H) or a mass status (35=AF), and contact the venue with the MsgSeqNums.

- **You reset with 141=Y.** A reset gives up every report not yet received; CumQty on the status answers shows the fills, not one by one (2.7).
- **A report twice.** After a resend, or after a venue crash, a report can arrive again; its ExecID is the same. Deduplicate by ExecID (2.8).
- **CumQty above the sum of the fills you have.** Fills could not be reported one by one; the Text on the report says so, and CumQty and LeavesQty are right (3.8).
- **An order with OrdStatusUnknown = Y.** Its fate is unknown; do not treat it as cancelled or rejected (4.7). Contact the venue to reconcile.
- **A rejected order you are sure was valid,** `exchange unavailable` or `not sequenced (gateway failover)`: the market was not reachable for a moment. The order did not trade; send it again under a new ClOrdID.

7.5 Orders and cancels refused

Read the Text: it is fixed and section 6 lists every one. The most common:

Text	Usually
`time in force 0, 1 or 6 only`	IOC or FOK sent: not supported (4.1)
`limit orders only`	OrdType other than 2
`unknown security`	Symbol misspelt, or 22 missing with 48. Take the identifiers from the security list (5.2)
`price not exact at 8 decimals`, `incorrect quantity`	A float formatted with more than eight decimals: format decimals exactly
`duplicate ClOrdID`	ClOrdIDs reused within the day; use unique ones
`ClOrdID must be 1-20 of A-Z a-z 0-9 - _ + ; : . /`	A ClOrdID too long, a UUID, or with another character: shorten it, or map your ids to ones that fit (4.8)
`MARKET_CLOSED`	The security is not trading: outside hours, or halted
`PRICE_OUT_OF_BOUNDS`	The price is outside the collar around the reference price
`ORDER_EXPIRED`	A GTD ExpireDate in the past
`too late to cancel`	The order was already filled, cancelled or expired
`already pending cancel`	Wait for the first cancel's answer

7.6 Market data

What you see	Likely cause
No 35=W after a 35=V	The venue is not yet sure of the book: the snapshot comes once it is (5.7). A 35=Y instead says why it was refused
A fresh 35=W in the middle of updates	A resynchronisation, or you fell behind (5.7). Replace your book with it
35=Y, MDReqRejReason 1	That MDReqID is already subscribed on the session; unsubscribe it or use another
35=y with SecurityRequestResult 4	The venue is starting up; ask again shortly
After a reconnect, nothing arrives	Subscriptions end with the session: subscribe again

7.7 Testing your connection

Before going live, in the venue's test environment:

1. Log on, log out, log on again, with sequence numbers carried over; then with a wrong password, and see `Logon refused`.
2. Enter, fill and cancel an order; check ExecType, OrdStatus, CumQty, LeavesQty and AvgPx on each report.
3. Log out with an order resting, have it filled, log on: the fill must arrive as a PossDup report your engine accepts.
4. Send an IOC, a market order and an unknown symbol: each must be rejected with its Text.
5. Send a 35=H and a 35=AF and reconcile your view against the answers.
6. Resend an order with PossDup = Y and check it is not executed twice.
7. Subscribe to a book at both depths, apply snapshots and updates, and compare with the venue's view; unsubscribe.
8. Reset with 141=Y and recover your orders from a 35=AF.

8. Index

References are to section numbers. **Bold** marks where a tag or term is defined.

8.1 Tags

6 AvgPx 3.8
8 BeginString 3.2
9 BodyLength 3.2
10 CheckSum 3.2
11 ClOrdID 1.4, 3.3, 3.4, 3.6, 4.8
14 CumQty 3.8, 2.7
15 Currency 5.2
17 ExecID 1.4, 3.8, 2.8
21 HandlInst 3.3
22 SecurityIDSource 1.3, **3.3**, 5.2
31 LastPx 3.8
32 LastQty 3.8
34 MsgSeqNum 2.3, 3.2
35 MsgType 3.2
37 OrderID 1.4, 3.8, 3.9
38 OrderQty 3.3, 3.8
39 OrdStatus 3.8, 4.4
40 OrdType 3.3, 4.1
41 OrigClOrdID 3.4, 3.9
43 PossDupFlag 2.5, 2.6, **3.2**, 4.8
44 Price 3.3
45 RefSeqNum 2.10, 3.10
48 SecurityID 3.3, 5.2
49 SenderCompID 2.2
52 SendingTime 2.4, **3.2**
54 Side 3.3
55 Symbol 3.3, 5.2
56 TargetCompID 2.2
58 Text section 6
59 TimeInForce 3.3, **4.2**
60 TransactTime 3.3, 3.8
97 PossResend 3.2
98 EncryptMethod 2.2
102 CxlRejReason 3.9, **6.3**
103 OrdRejReason 3.8, **6.1**, 6.2
107 SecurityDesc 5.2
108 HeartBtInt 2.2, 2.4
112 TestReqID 2.4
122 OrigSendingTime 2.5, **3.2**
123 GapFillFlag 2.5
141 ResetSeqNumFlag 2.2, **2.7**
146 NoRelatedSym 5.2, 5.3
150 ExecType 3.8
151 LeavesQty 3.8
262 MDReqID 5.3
263 SubscriptionRequestType 5.3
264 MarketDepth 5.3, 5.5
265 MDUpdateType 5.3
266 AggregatedBook 5.3
267 NoMDEntryTypes 5.3
268 NoMDEntries 5.4, 5.5
269 MDEntryType 5.3, **5.4**, 5.5
270 MDEntryPx 5.4, 5.5
271 MDEntrySize 5.4, 5.5
279 MDUpdateAction 5.5
281 MDReqRejReason 5.6
286 OpenCloseSettlFlag 5.3
320 SecurityReqID 5.2
322 SecurityResponseID 5.2
336 TradingSessionID 5.3
346 NumberOfOrders 5.4, 5.5
371 RefTagID 2.10
372 RefMsgType 2.10, 3.10
373 SessionRejectReason 2.10, **6.4**
380 BusinessRejectReason 3.10, **6.5**
393 TotNoRelatedSym 5.2
432 ExpireDate 3.3, 4.2
434 CxlRejResponseTo 3.9
454 NoSecurityAltID 5.2
455 SecurityAltID 5.2
456 SecurityAltIDSource 5.2
546 Scope 5.3
547 MDImplicitDelete 5.3
553 Username 2.2
554 Password 2.2
559 SecurityListRequestType 5.2
560 SecurityRequestResult 5.2
584 MassStatusReqID 3.7
585 MassStatusReqType 3.7
790 OrdStatusReqID 3.6

893 LastFragment 5.2
911 TotNumReports 3.7
912 LastRptRequested 3.7
969 MinPriceIncrement 3.11, 5.2
2362 SelfMatchPreventionID 3.3, 3.11, 4.3
2964 SelfMatchPreventionInstruction 3.3, 3.11, 4.3
5001 OrdStatusUnknown 3.11, 4.7

8.2 Messages

MsgType	Name	Direction	Section
0	Heartbeat	Both	2.4
1	TestRequest	Both	2.4
2	ResendRequest	Both	2.5
3	Reject	Both	2.10, 6.4
4	SequenceReset	Both	2.5
5	Logout	Both	2.9, 6.6
A	Logon	Both	2.2
D	NewOrderSingle	To the venue	3.3
F	OrderCancelRequest	To the venue	3.4
G	OrderCancelReplaceRequest	To the venue, refused	3.5
H	OrderStatusRequest	To the venue	3.6
AF	OrderMassStatusRequest	To the venue	3.7
8	ExecutionReport	From the venue	3.8
9	OrderCancelReject	From the venue	3.9
j	BusinessMessageReject	From the venue	3.10, 6.5
x	SecurityListRequest	To the venue	5.2
y	SecurityList	From the venue	5.2
V	MarketDataRequest	To the venue	5.3
W	MarketDataSnapshotFullRefresh	From the venue	5.4
X	MarketDataIncrementalRefresh	From the venue	5.5
Y	MarketDataRequestReject	From the venue	5.6

8.3 Terms

Cancel on disconnect, none 2.9, 4.5
ClOrdID characters and length 1.4, 3.3, 4.8, 6.1
ClOrdID uniqueness 1.4, 4.8
Day order 4.2
Decimals, exact to eight places 1.3, 7.5
Deduplicating by ExecID 2.6, 2.8
Gap fill 2.5, 2.8
Gateway failover 4.7
GTC, GTD 4.2
Logon refused 2.2, 7.2
Mass status after a reset 2.7
Offline delivery of reports 2.6
Order states 4.4
Participant 1.2

Resetting sequence numbers 2.7
Restart of the venue 2.8
Self-match prevention 4.3
Sequence numbers 2.3
Slow consumer 5.7
Testing your connection 7.7
Transient session 2.3, 5.1
Unknown status 4.7, 6.3
Volatility halt 4.5, 6.2